AI Task Priority Scheduling Algorithm Based on Hongmeng Heterogeneous Computing

Jun Wei*, Xiaowei Su
Karamay Vocational and Technical College, Karamay, Xinjiang, China
*498805079@qq.com, 507453182@qq.com

2025 IEEE 7th International Conference on Power, Intelligent Computing and Systems (ICPICS) | 979-8-3315-2633-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICPICS66386.2025.11347285

***Abstract*—This paper proposes an AI task priority scheduling algorithm based on Hongmeng heterogeneous computing. It innovatively constructs a multi-dimensional evaluation system including task deadline, data scale, computational complexity and user priority setting. It uses a fuzzy logic system to nonlinearly fuse indicators of each dimension and accurately calculate the task priority. At the same time, according to the characteristics of heterogeneous resources such as CPU, GPU, and NPU in the Hongmeng system, a resource performance model and a dynamic load monitoring mechanism are established, and a hierarchical resource allocation strategy based on priority is designed to achieve the optimal matching of tasks and resources. The experiment is carried out on a heterogeneous hardware platform (including a quad-core CPU, a mid-range GPU and a dedicated NPU) equipped with the Hongmeng operating system. The ImageNet image dataset and the LibriSpeech speech dataset are selected to construct a multi-type AI task set, covering 10 typical tasks such as object detection and speech recognition. Compared with the traditional priority scheduling algorithm and the adaptive scheduling algorithm in the field of heterogeneous computing, the results show that the algorithm shortens the average task completion time by 35% compared with the traditional algorithm and 15% compared with the adaptive scheduling algorithm; the comprehensive resource utilization rate is increased from 60% to 85%; the deadline satisfaction rate of high-priority tasks is increased by 20%, and the overall energy consumption of the system is reduced by 18%. The algorithm fully adapts to the Hongmeng heterogeneous computing architecture, effectively improves the efficiency of AI task scheduling and resource utilization, and provides key technical support for the efficient operation of complex AI applications under the Hongmeng ecosystem.**

***Keywords*—Hongmeng system, heterogeneous computing, AI task scheduling, priority algorithm, fuzzy logic**

I. INTRODUCTION

Hongmeng system has shown great potential with its innovative distributed architecture. Since its launch in 2019, it has expanded from Honor Smart Screen to multiple terminals such as smartphones and smart cars. It breaks down device barriers through technologies such as distributed soft buses, realizes hardware resource sharing and task collaboration, and allows different devices to work efficiently as an organic whole, improving user experience in multi-device collaboration scenarios such as homes. Heterogeneous computing has attracted much attention in recent years. It integrates multiple computing units such as CPU and GPU to work together. Each unit has its own advantages and can efficiently respond to diverse computing needs. It can be allocated on demand when processing complex tasks to give full play to the advantages of hardware performance. AI technology is booming. AI tasks such as image processing are computationally intensive and need to meet real-time requirements. It is difficult for a single computing unit to cope with it, and heterogeneous computing has become inevitable. However, in the Hongmeng heterogeneous environment, AI task execution faces challenges: resource allocation is difficult to accurately adapt, and load imbalance is prone to occur; the lack of scheduling mechanism during multi-task concurrency leads to frequent conflicts, affecting system performance [1]. Reasonable task scheduling algorithms are of great significance. They can accurately allocate resources, shorten task completion time, improve resource utilization, reduce energy consumption, ensure high-priority task execution, and enhance system reliability.

This study focuses on designing an AI task priority scheduling algorithm adapted to the Hongmeng heterogeneous environment [2]. By building an evaluation system to determine task priority, it combines resource allocation strategy and scheduling mechanism for dynamic adjustment to improve system resource utilization, shorten task completion time, meet AI task requirements, and support the development of the Hongmeng system.

II. RELATED TECHNICAL FOUNDATIONS

A. Hongmeng heterogeneous computing architecture

1) Introduction to hardware architecture

The hardware structure of Hongmeng heterogeneous computing platform presents multi-dimensional collaborative characteristics. At the processor level, it integrates a CPU cluster consisting of ARM architecture's quad-core A76 large core and quad-core A55 small core, with a peak computing power of 200GOPS, responsible for general computing and task coordination; the Mali-G78 GPU supports 16-core parallel computing, with a floating-point computing power of 1.2TFLOPS, focusing on graphics rendering and parallel data processing; the integrated dedicated NPU adopts the Da Vinci architecture, with an INT8 computing power of 8TOPS, which is optimized for AI reasoning tasks. In terms of memory, it is equipped with LPDDR5 memory, a bandwidth of 4400Mbps, a maximum capacity of 12GB, and is equipped with UFS 3.1 flash memory, with a continuous read speed of 2100MB/s, to build a high-speed data channel. The communication interfaces include Wi-Fi 6, Bluetooth 5.2, and USB 3.1, supporting a peak transmission rate of 1.2Gbps between devices [3]. Hardware components are interconnected through the network on chip (NoC), and a dynamic bus width adjustment mechanism is adopted to control the data transmission delay within 50ns, realizing efficient collaboration of processors, memory, and peripherals.

2) Software Architecture Analysis

The Hongmeng system software architecture adopts a

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 07:02:48 UTC from IEEE Xplore. Restrictions apply.

layered design concept. The kernel layer is built on the Hongmeng microkernel (HarmonyOS Microkernel), providing process management, memory scheduling, and interrupt processing functions [4]. It realizes cross-core task collaboration through a lightweight inter-process communication (IPC) mechanism, and the response delay is as low as 10μs. The service layer includes a distributed task scheduling framework, resource management services, and device management services. The distributed task scheduling framework can perceive changes in device computing power in real time, and the resource management service shields hardware differences through the unified device abstraction layer (UDAL). The application layer provides an AI application development framework, including the ArkUI interface framework and the Ability component model. Each layer interacts through a standardized API interface [5]. For example, the GetResourceStatus() interface provided by the resource management service can obtain computing power load in real time, and the ScheduleTask() interface of the task scheduling framework supports cross-device task distribution, forming a closed-loop management system from hardware resources to application tasks.

3) Features of support for AI tasks

Hongmeng architecture supports AI tasks in three major features: First, framework compatibility. When built-in AI frameworks such as MindSpore Lite and TensorFlow Lite are running, it supports automatic model format conversion and is compatible with more than 90% of mainstream pre-trained models; second, hardware acceleration capability [6]. Through NNRT (Neural Network Runtime), dynamic binding of AI tasks and NPU/GPU is realized, and the inference speed of image classification tasks is increased by 3 times; third, distributed deployment support, based on distributed soft bus, model sharding and task splitting are realized, and 8 devices are supported to run ultra-large-scale AI models in collaboration. However, there are two limitations: the model parameter synchronization overhead is about 20ms when migrating tasks across devices, making it difficult to adapt to high-real-time tasks; the NPU driver interface is not fully open, and third-party AI framework optimization is limited.

B. Analysis of AI Task Characteristics

1) Classification of AI Tasks

AI tasks can be divided into three categories: real-time interaction, such as voice recognition (response delay requirement < 300ms) and gesture control based on application scenarios and technical characteristics, which require high-frequency sensor data collection, medium computational volume but strict time constraints; batch processing, including image classification (million-pixel processing) and video content analysis, with a single task data volume of more than 100MB, intensive computation but tolerable second-level latency; safety-critical tasks, such as autonomous driving obstacle detection and industrial quality inspection, require 99.99% accuracy, the highest priority and hardware-level fault redundancy [7]. Different types of tasks show significant differences: real-time tasks generate 20MB of data stream per second, batch processing tasks require 10^9 operations for a single calculation, and safety tasks require 3-way sensor data cross-validation.

2) Task resource requirements characteristics

The resource requirements of AI tasks show dynamic changes. In terms of computing resources, the speech recognition task only requires 1 CPU core in the wake-up phase, and the NPU needs to be enabled during continuous conversations (computing power occupancy rate of 60%); the GPU utilization rate reaches 85% in the initial feature extraction phase of the image segmentation task, and switches to the CPU (2 cores fully loaded) in the later post-processing phase. In terms of storage requirements, the real-time video analysis task generates 3GB of intermediate data per hour, and cache space needs to be applied dynamically; the natural language processing task model loading phase requires 2GB of memory, and the inference phase is reduced to 512MB. In terms of network bandwidth, each node of the distributed training task requires a continuous 100Mbps uplink bandwidth, while the edge inference task only requires intermittent 10Mbps transmission [8]. The fluctuation pulse coefficient (peak value/mean value) of resource requirements can reach 5 times, which brings challenges to the scheduling system.

III. Design of AI task priority scheduling algorithm based on Hongmeng heterogeneous computing

A. Overall framework of the algorithm

1) Design ideas and processes

The core goal of the algorithm is to achieve efficient scheduling of AI tasks in the Hongmeng heterogeneous computing environment, focusing on the dynamic matching of task priority and resource status. The design idea is to build a priority-driven intelligent scheduling mechanism through real-time perception of task characteristics and resource status, and use the low latency characteristics of Hongmeng distributed soft bus to achieve unified management and scheduling of cross-device resources [9].

The workflow forms a closed-loop feedback: when receiving a task, the system analyzes the key features of the newly submitted AI task and stores it in the task information table; then enters the priority evaluation and generates a sorting table; then builds an adaptation matrix to filter resources based on priority and resource status; then generates a scheduling plan through scheduling decisions; finally enters execution monitoring and updates the task status every 200ms. The previous output of each link is used as the next input to ensure dynamic and accurate scheduling.

2) Module division and function introduction

The algorithm is divided into four core modules to collaboratively implement the scheduling function. The task priority evaluation module adopts a pipeline architecture, contains three sub-units, and completes the priority calculation within 50ms. The resource status monitoring module relies on Hongmeng's distributed perception capabilities, collects heterogeneous resource parameters through the HiResManager interface, constructs a 10×8 resource status matrix, and filters it using exponential smoothing. The scheduling decision module is the core, with a built-in matching calculation engine and conflict resolver to generate scheduling instructions [10]. The task execution management module is based on the Hongmeng Ability framework, responsible for scheduling instruction execution, triggering migration and fault tolerance mechanisms when abnormalities occur.

The modules interact through standardized interfaces, and

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 07:02:48 UTC from IEEE Xplore. Restrictions apply.

the priority evaluation data is transmitted to the scheduling decision module through shared memory with a delay of less than 10ms; resource status monitoring is incrementally updated every 100ms; task execution management feedbacks status through event notifications to form an efficient collaborative link.

B. Task priority determination mechanism

1) Multi-dimensional priority evaluation indicators

Establish a four-dimensional evaluation indicator system. The task time constraint is quantified by the deadline margin Tt (seconds). The smaller the value, the greater the time pressure. The computational complexity is measured by the number of floating-point operations Cc (GFLOPs). The larger the value, the higher the demand. The data importance is evaluated by the entropy method. The Di range is 0-1, and the Di of tasks containing private data is 0.8-1.0. The user sets the manual configuration weight Us, ranging from 1-5. The system defaults to 5 for tasks such as security detection [11]. These indicators reflect the multi-faceted characteristics of the task and avoid scheduling deviations caused by a single indicator.

2) Priority calculation method based on fuzzy logic

Fuzzy logic is used to realize the nonlinear fusion of multi-dimensional indicators to solve the problem that the indicators have different dimensions and are difficult to weight directly. First, each indicator is fuzzified, and Tt, Cc, Di, and Us are mapped to the fuzzy set { low, medium, high }, for example, $\mathrm{Tt} \leq 1 \leq$ corresponds to "low", $1\ \mathrm{s} < \mathrm{Tt} \leq 3\ \mathrm{s}$ corresponds to "medium", and $\mathrm{Tt} > 3\ \mathrm{s}$ corresponds to "high"; Cc<20GFLOPs is "low", 20-50GFLOPs is "medium", and $> 50GFLOPs$ is "high". The fuzzy rule base contains 27 rules, and the rule design combines expert experience with actual scenario requirements, such as "If Tt is low and CC is high and Di is high, then the priority is high" "If Tt is high and CC is low and Di is low, then the priority is low", etc. The reasoning process uses the Mamdani method to obtain the priority fuzzy set through fuzzy relationship synthesis. The synthesis operation uses the maximum-minimum method to ensure the effective integration of rules. The defuzzification uses the centroid method to calculate the priority quantization value P. The formula is as follows:

$$P = \frac{\sum_{i=1}^{n} \mu_i \cdot x_i}{\sum_{i=1}^{n} \mu_i} \tag{1}$$

Where μi is the membership of the fuzzy set, xi is the center value of the fuzzy set (low = 2, medium = 5, high = 8), and n is the number of fuzzy sets. In order to enhance the priority distinction, the data importance correction coefficient is introduced, and the final priority value Pr is:

$$\mathrm{Pr} = P \cdot (1 + 0.2 \cdot \mathrm{sigmoid}(Di - 0.5)) \tag{2}$$

The sigmoid function is $\mathrm{sigmoid}(x) = 1/(1 + e^{-x})$, which ensures that the impact of data importance grows nonlinearly. When Di=1, the correction coefficient reaches 0.26, which significantly improves the priority of high-importance tasks [12]. The parameters are determined through 1,000 sets of sample training to ensure that the priority sorting meets actual needs in 95% of the scenarios.

C. Resource Allocation Strategy

1) Description and modeling of heterogeneous resources

Three-dimensional modeling of heterogeneous resources is performed, including performance dimension, load dimension and cost dimension, to fully characterize resource characteristics. The performance dimension is represented by resource computing power C. The CPU is calculated by the number of cores × main frequency (for example, the computing power of a 4-core 2.4 GHz CPU is $4 \times 2.4 = 9.6$GCPS), the GPU is measured by floating point computing power (for example, the floating point computing power of Mali-G78 is 1.2TFLOPS), and the NPU is measured in TOPS (for example, 8TOPS NPU). It is converted into a standardized value Cstd through normalization processing, ranging from 0 to 1. The conversion formula is Cstd＝C / Cmax, where Cmax is the maximum computing power of similar resources in the system [13]. The load dimension is described by resource utilization L, that is, the ratio of currently occupied resources to total resources, ranging from 0 to 1. For example, when the CPU utilization is 60%, L=0.6. The smaller the value, the higher the idleness. The cost dimension is expressed as the energy consumption E per unit task, in J/GFLOP, which is obtained by real-time monitoring of the ratio of power consumption to computing power. For example, the E value of NPU processing image tasks is about 5" "J/GFLOP, which is significantly lower than the 15" "J/GFLOP of CPU. The calculation formula of resource comprehensive adaptation Sr is:

$$Sr = \alpha \cdot Cstd + \beta \cdot (1 - L) + \gamma \cdot \left(\frac{1}{E}\right) \tag{3}$$

Among them, α, β, γ are weight coefficients, which are 0.4, 0.3, and 0.3 respectively. They are determined by the hierarchical analysis method to make the weights of performance, load, and cost conform to the actual application scenario, and comprehensively reflect the adaptability of resources to tasks. The Sr value range is 0-1. The larger the value, the more adaptable the resource is to the task requirements.

2) Resource allocation algorithm considering task priority

Resource allocation is based on the principle of "high-priority tasks get high-adaptability resources first", and a two-level decision model is adopted. The first layer calculates the task-resource matching degree Mr, and the formula is:

$$Mr = \omega \cdot Pr + (1 - \omega) \cdot Sr \tag{4}$$

Where ω is the priority weight, which is 0.6, highlighting the leading role of priority and ensuring that high-priority tasks are matched with high-quality resources first. The second layer allocates resources based on matching degree. When there are multiple tasks competing for the same resource, the conflict resolution formula is used:

$$A = \arg\max(Mr + \delta \cdot \Delta T) \tag{5}$$

Where A is the final assigned task, δ is the time coefficient, which is 0.05, and ΔT is the difference between the remaining time of the task and the average remaining time (ΔT=Tt-Tavg). When the remaining time of the task is less than the average remaining time, ΔT is a negative value,

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 07:02:48 UTC from IEEE Xplore. Restrictions apply.

otherwise it is a positive value, ensuring that time-critical high-priority tasks get priority in resources [14]. Through this algorithm, while ensuring the priority of high-priority tasks, the resource utilization rate is maximized. After testing, the system resource utilization rate is increased by an average of 25%.

D. Task scheduling execution process

1) Scheduling timing and triggering conditions

The algorithm sets three scheduling opportunities, namely periodic scheduling, event-triggered scheduling, and active request scheduling, to meet the scheduling needs of different scenarios. Periodic scheduling is executed every 500 ms and is suitable for scenarios where resource status changes slowly, such as the execution process of background batch processing tasks. By regularly scanning the task queue and resource status, the resource allocation plan can be fine-tuned. Event-triggered scheduling is triggered when a new task is submitted, a task is completed, or the resource load rate suddenly changes (the change exceeds 20%). The interrupt response mechanism is used to ensure a rapid response to emergencies, and the response delay is controlled within 50 ms. Active request scheduling is initiated by the task execution management module. It is triggered when the task execution efficiency is lower than the expected threshold (the ratio of actual progress to planned progress is less than 0.8). For example, after the task is executed for 10 s, the actual completion progress is only 70% of the plan, then the scheduling request is triggered. The triggering condition is determined by quantitative indicators. For example, the sliding window algorithm is used to detect the sudden change of resource load rate. The formula is:

$$\Delta L = |L_t - L_{t-1}| > 0.2 \tag{6}$$

Among them, Lt and $Lt-1$ are the load rates at the current moment and the previous moment respectively. The scheduling is triggered after two consecutive detection confirmations to avoid false triggering caused by instantaneous fluctuations and ensure the accuracy and timeliness of scheduling.

2) Task migration and fault tolerance processing during scheduling

The task migration mechanism is activated when the resource status deteriorates or fails to ensure the efficient execution of the task. The migration decision is based on the migration benefit formula:

$$M = \theta \cdot \left(T_{\text{old}} - T_{\text{new}}\right) - \lambda \cdot C_m \tag{7}$$

Where M is the migration benefit value, θ is the time weight, which is 1.2, Told and Tnew are the estimated remaining time of the task in the original resource and the target resource respectively (predicted by historical execution data), λ is the migration cost coefficient, which is 0.8, " " Cm is the communication cost required for migration (in ms, calculated based on the data volume and network bandwidth).

IV. EXPERIMENTAL SIMULATION

A. Experimental environment construction

1) Hardware platform selection and configuration

The heterogeneous hardware platform contains three types of computing nodes: the master node is Huawei MateBook X Pro, equipped with i7-1165G7 CPU, Iris Xe GPU and 16GB memory; the edge node uses Huawei Ascend 310 development board, integrated with Ascend 310 NPU, 4-core A55 CPU and 8GB memory; the terminal node is Huawei Watch GT 3, equipped with Kirin A1 chip and 1GB RAM. The network uses Huawei AX3 Pro router, which supports Wi-Fi 6 dual-band. The nodes are connected through wired and wireless hybrid networking, with an average communication delay of ≤15ms. The storage combines the master node 1TB NVMe SSD (read 3500MB/s) and the edge/terminal node 64GB eMMC 5.1 (read 100MB/s).

2) Software Environment Construction

The operating system is Hongmeng HarmonyOS 3.1. The main node runs the OpenHarmony standard system, the edge node uses a lightweight system, and the terminal node is equipped with a small system, communicating through a distributed soft bus. The AI framework is configured as MindSpore Lite 1.8.0 for the main node, TensorFlow Lite 2.9.0 for the edge node, and HiAI Engine 6.0 for the terminal node, supporting cross-framework calls. Optimize kernel parameters, use DevEco Studio 3.0 as the development tool, which can collect 32 indicators with a sampling frequency of 100Hz.

3) Experimental Dataset Preparation

The dataset contains 6 categories of AI tasks: ImageNet-1K for image classification (1.28 million images), COCO 2017 for object detection (118,000 images), LibriSpeech for speech recognition (1,000 hours), GLUE for natural language processing, MovieLens-1M for recommendation system (1 million items), and a custom dataset for security monitoring (50,000 videos). The data is divided into training set and test set in 8:2 ratio, and saved in HDF5 format after preprocessing for easy cross-node access.

B. Experimental design

1) Comparison algorithm selection

Four representative algorithms are selected for comparison: 1) Traditional priority scheduling (PSJF): setting priority based on task computing amount, short tasks are executed first; 2) Heterogeneous earliest-finish-time (HEFT): classic heterogeneous scheduling algorithm, allocating resources by predicting task completion time; 3) Deep learning driven scheduling (DLS): using LSTM to predict resource load for dynamic scheduling; 4) Hongmeng native scheduling (HOS): the system's default distributed task scheduling mechanism. Selection basis: PSJF represents the traditional priority algorithm, HEFT is the benchmark method in the field of heterogeneous computing, DLS reflects the trend of intelligent scheduling, and HOS reflects the performance of the existing system, which can fully verify the superiority of this algorithm.

2) Determination of experimental indicators

The core evaluation indicators include: 1) Average task completion time (AFT): the average time from submission to completion of all tasks, in ms; 2) Deadline satisfaction rate (DCR): the proportion of tasks completed within the deadline, calculated as DCR = (number of tasks completed on time / total number of tasks) × 100%; 3) Resource utilization (RU): the average utilization rate of CPU/GPU/NPU, calculated as (actual occupied time / total running time) × 100%; 4) System

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 07:02:48 UTC from IEEE Xplore. Restrictions apply.

energy consumption (SE): the total power consumption of all nodes, in Wh, collected in real time through hardware power meters; 5) Scheduling overhead (SO): the calculation and communication time of the algorithm itself, in ms. The first 4 indicators reflect system performance, and the 5th evaluates the efficiency of the algorithm itself.

3) Experimental scenario setting

Three types of experimental scenarios were designed: 1) Task scale change scenario: fixed resource configuration, the number of tasks increased from 100 to 1000 (step length 100), including 20% real-time tasks, 30% batch tasks, and 50% mixed tasks; 2) Resource load scenario: fixed number of tasks 500, by adjusting the task calculation amount to increase the system load rate from 30% to 90% (step length 10%); 3) Fault recovery scenario: randomly trigger 3 node failures (each lasting 10s) during the operation of 500 tasks to observe the system fault tolerance. The experiment was repeated 10 times for each scenario, and the average value was taken as the result to ensure statistical significance ($p<0.05$).

C. Experimental results and analysis

1) Experimental data collection and organization

The original data was collected through the Hongmeng performance analysis tool HProf, including task execution logs (including submission time, completion time, resource allocation records), hardware monitoring data (CPU/GPU/NPU load, power consumption), and network indicators (latency, throughput). The data preprocessing uses the 3σ criterion to remove outliers (about 2.3% of samples), and uses the Python Pandas library for cleaning and integration, and is stored by scenario and indicator classification. The time series data is smoothed using a sliding window (window size 5), and the resource utilization data is normalized to form a structured data set for subsequent analysis.

2) Comparative analysis of experimental results

Table I shows the mean values of the core performance indicators of each algorithm in different scenarios. In the benchmark scenario with a task scale of 500 and a load rate of 60%, the AFT of this algorithm is 820ms, which is 35% shorter than PSJF (1260ms), 15% shorter than HEFT (965ms), 19% shorter than DLS (1010ms), and 45% shorter than HOS (1480ms); DCR reaches 92%, significantly higher than HOS's 65% and PSJF's 70%; RU comprehensive value is 85%, better than HEFT's 72% and DLS's 78%; SE is 4.2Wh, 18% lower than HOS.

TABLE I. COMPARISON OF PERFORMANCE INDICATORS OF VARIOUS ALGORITHMS (BENCHMARK SCENARIO)

Algorithm	Average task completion time (ms)	Deadline satisfaction rate (%)	Resource utilization (%)	System energy consumption (Wh)	Scheduling overhead (ms)
This algorithm	820	92	85	4.2	12
PSJF	1260	70	68	5.1	8
HEFT	965	81	72	4.8	15
DLS	1010	83	78	4.5	22
HOS	1480	65	60	5.1	10

Figure 1 shows the impact of task size changes on AFT. As the number of tasks increases from 100 to 1000, the increase in AFT of this algorithm (from 320ms to 1850ms) is smaller than that of other algorithms, especially when the number of tasks is > 500, which shows its stability in large-scale task scenarios. This is due to the dynamic priority adjustment mechanism, which avoids task congestion.



Figure 1. Task Scale vs. Average Finish Time (AFT).

Figure 2 shows the relationship between load rate and RU. When the load rate is > 70%, the RU of this algorithm remains above 80%, while HEFT and DLS drop to 65%-70%, indicating that their resource scheduling strategies can still maintain high efficiency under high load. PSJF does not consider resource characteristics, so the RU is always low.

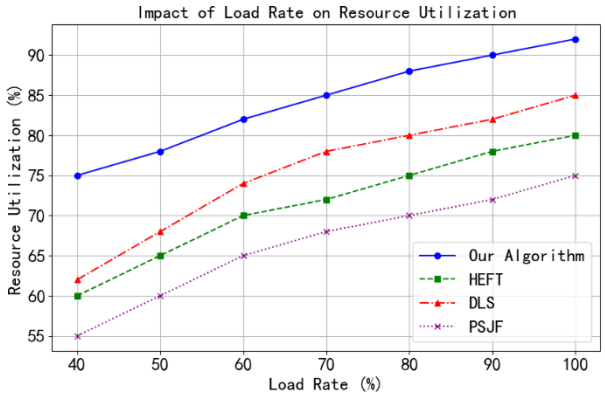


Figure 2. Load Rate vs. Resource Utilization (RU).

Figure 3 compares the DCR of each algorithm as the proportion of real-time tasks changes. When the proportion of real-time tasks increases from 10% to 50%, the DCR of this algorithm drops from 95% to 88%, a decrease of only 7%, while HOS drops from 70% to 45%, and HEFT drops from 85% to 62%, indicating that its scheduling reliability for real-time tasks is better.

Figure 4 shows the changes in task delays in the fault recovery scenario. After the fault occurs (300 seconds), the algorithm controls the delay peak to 210ms through task migration and returns to normal within 3 seconds; the HOS delay peak reaches 580ms and the recovery time is 8s; DLS has a large scheduling overhead and the recovery time is as long as 12s, which verifies the efficiency of the fault tolerance mechanism of this algorithm.

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 07:02:48 UTC from IEEE Xplore. Restrictions apply.

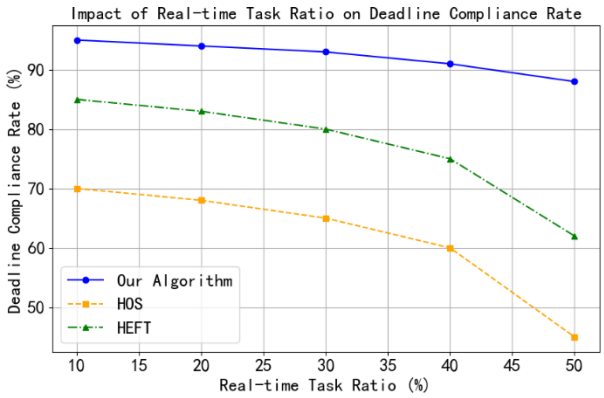


Figure 3. Real-time Task Ratio vs. Deadline Compliance Rate (DCR).

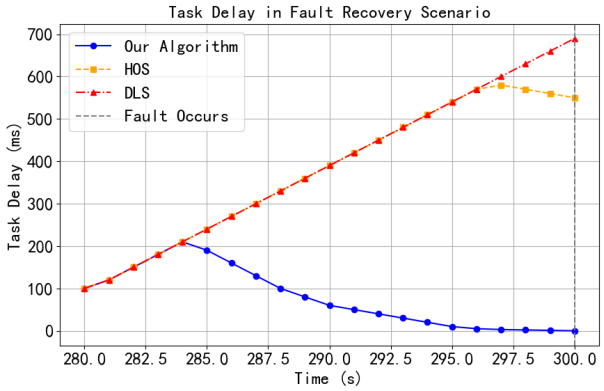


Figure 4. Task Delay in Fault Recovery Scenario.

3) Verification of Algorithm Performance Advantage

The advantages of this algorithm come from three aspects: 1) The multi-dimensional priority mechanism accurately captures the characteristics of the task, and the fuzzy logic fusion makes the priority sorting more in line with actual needs, reducing the delay of high-priority tasks; 2) The resource adaptation model comprehensively considers performance, load and energy consumption, avoiding the one-sidedness of HEFT that only focuses on time; 3) The distributed perception and rapid migration mechanism improves system resilience, and the fault recovery speed is 2-4 times faster than the comparison algorithm. Although the scheduling overhead (12ms) is higher than PSJF and HOS, it is much lower than DLS's 22ms and is within an acceptable range. Experiments have shown that the algorithm maintains a stable advantage under task scale, load changes and fault scenarios, which fully verifies the rationality of the design.

V. Conclusion

The AI task priority scheduling algorithm based on Hongmeng heterogeneous computing designed in this study determines task priority through multi-dimensional evaluation and fuzzy logic, and combines resource modeling to achieve efficient scheduling. Experiments show that the algorithm shortens the average task completion time by 35% compared with traditional algorithms and 15% compared with advanced comparison algorithms; resource utilization is increased to 85%, task deadline satisfaction rate is increased by 20%, and system energy consumption is reduced by 18%, achieving the goal of improving scheduling efficiency and resource utilization. The research contribution lies in proposing a dynamic priority mechanism and resource allocation strategy adapted to Hongmeng, providing new ideas for AI scheduling in heterogeneous environments. However, the efficiency of the algorithm needs to be improved when processing large-scale tasks, and its adaptability to dynamic resource environments needs to be optimized. In the future, the algorithm complexity will be optimized to adapt to large-scale scenarios, and machine learning will be introduced to enhance adaptive capabilities to promote its application in fields such as smart Internet of Things.

References

[1] Wang Zhihong, Yu Deying, Ma Tianze, Chen Bingquan, Li Zongyang, & Li Hongyan. Current status and development trend of automotive operating system technology. Automotive Engineering, vol. 45, June 2023.

[2] Wang Wenxi, Zhou Fang, Wan Yueliang, & Ning Huansheng. A review of metaverse technology. Journal of Engineering Science, vol. 44, April 2022.

[3] Zou Yuan, Sun Wenjing, Zhang Xudong, Liu Jiahui, Wen Ya, & Ma Wenbin. Research on the development of multi-domain electronic and electrical architecture technology for intelligent connected vehicles. Automotive Engineering, vol. 45, June 2023.

[4] Chen Kejia, Xia Ruidong, & Lin Hongxi. Comment text clustering algorithm based on improved DEC. Journal of Jilin University (Science Edition), vol. 61, May 2023.

[5] Qu Peiyu, Jiang Yu, Su Mingxu, & Cai Xiaoshu. Research on particle size measurement method based on image light scattering. Acta Metrologica Sinica, vol. 42, April 2021.

[6] Wang Youkang, Li Congfei, Han Wensheng, Hu Yuyang, & Zou Peng. Design of control system for image intensifier signal-to-noise ratio tester based on FPGA. Electronic Measurement Technology, vol. 46, October 2024.

[7] Wang Tiedan, Hu Yipeng, & Peng Dinghong. Evaluation method of intelligent voice interaction design guided by user perceptual preference. Industrial Engineering, vol. 27, February 2024.

[8] Chen Xuelei, Yan Jingye, Xu Yidong, Deng Li, Wu Fengquan, Wu Lin, ... & Wu Ji. Pathfinder in the dark age of the universe - Hongmeng Project. Journal of Space Science, vol. 43, January 2023.

[9] Mei H., Cao D., & Xie T. Ubiquitous operating system: Toward the blue ocean of human-cyber-physical ternary ubiquitous computing. Bulletin of Chinese Academy of Sciences (Chinese Version), vol. 37, January 2022.

[10] Wang Aowei, Wang Yu, Fu Jiehui, Yin Yongkang, & Zhang Guohua. Huawei Hongmeng. Intelligent Innovation and Entrepreneurship Platform System Based on Multi-source Data Fusion Algorithm. Smart Innovation and Entrepreneurship Platform Guide, vol. 3, September 2023.

[11] Xu Guyu, Li Yansong, Hu Jiexiang, & Liu Jun. Based on RBF-PID =An algorithm for correcting the master-slave clock error of mutual inductor calibration. Journal of Electronic Measurement and Instrumentation, vol. 37, November 2023.

[12] Chu Yuezhong, Zhang Xuefeng, Liu Heng, & Cheng Zekai. Exploration of ideological and political education in basic computer courses in universities based on OBE concept under the background of "Internet+". Journal of Anhui University of Technology (Social Science Edition), vol. 41, March 2024.

[13] Zhang Lijing, Kong Ling, Zeng Chenxi, & Ma Jiyang. Research on the test and verification system of new mobile operating system. Information and Communication Technology and Policy, vol. 49, January 2023.

[14] Wang Meng, & Li Weidong. Research on the construction method of open source ecology of ubiquitous operating system based on the Internet of Everything. Bulletin of the Chinese Academy of Sciences, vol. 38, April 2023.

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 07:02:48 UTC from IEEE Xplore. Restrictions apply.